

Deep Learning Practical 7

Flow matching: from points to speech

1 Move noise into eight clusters

We will train a network to move points from a Gaussian distribution into eight clusters, then apply the same construction to speech spectrograms. The two-dimensional example lets us plot the generated distribution and follow individual trajectories.

Our data distribution has eight small Gaussian clusters around a circle. The function below draws a fresh batch whenever we call it:

```
import torch
from torch import nn
from torch.nn import functional as F
import matplotlib.pyplot as plt

torch.manual_seed(3801)
device = "cuda" if torch.cuda.is_available() else "cpu"

def sample_data(n):
    label = torch.randint(0, 8, (n,), device=device)
    angle = 2*torch.pi*label/8
    centres = 2*torch.stack((angle.cos(), angle.sin()), dim=1)
    return centres + .12*torch.randn(n, 2, device=device)
```

Choose an independent noise point $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, I)$ and data point $\mathbf{x}_1 \sim p_d$. A straight path between them has position and velocity

$$\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \frac{d\mathbf{x}_t}{dt} = \mathbf{x}_1 - \mathbf{x}_0, \quad t \in [0, 1].$$

For example, the path from $(0, 0)$ to $(2, 0)$ passes through $(1, 0)$ at $t = 1/2$, with velocity $(2, 0)$. The network takes the current two coordinates and time as its three inputs, and returns two velocity components.

```
class Velocity2D(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(3, 64), nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, 2))

    def forward(self, x, t):
        return self.net(torch.cat((x, t[:], None]), dim=1))
```

We train by averaging squared error over both coordinates:

$$\mathcal{L}(\theta) = \mathbb{E}_{\substack{\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, I), \mathbf{x}_1 \sim p_d \\ t \sim \mathcal{U}(0,1)}} \left[\frac{1}{2} \|v_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|_2^2 \right].$$

Here $\mathbf{x}_t$ and its target velocity can be calculated directly from the sampled pair. Different pairs can pass through similar intermediate states with different velocities; squared-error regression learns their conditional mean at a given state and time.

```

velocity = Velocity2D().to(device)
optimiser_v = torch.optim.Adam(velocity.parameters(), lr=.001)
velocity_history = []
for step in range(8000):
    x1 = sample_data(128)
    x0 = torch.randn_like(x1)
    t = torch.rand(len(x1), device=device)
    xt = (1-t[:, None])*x0 + t[:, None]*x1
    loss = F.mse_loss(velocity(xt, t), x1-x0)
    optimiser_v.zero_grad(set_to_none=True)
    loss.backward()
    optimiser_v.step()
    velocity_history.append(loss.item())

```

Exercise

Check the path endpoints and explain why this training loop needs no numerical integration. If we instead choose $\mathbf{x}_t = (1 - t^2)\mathbf{x}_0 + t^2\mathbf{x}_1$, what should replace the target $\mathbf{x}_1 - \mathbf{x}_0$?

2 Generate points by following the learned field

Starting with fresh Gaussian noise, Euler integration repeatedly adds the predicted velocity multiplied by the step length. The network receives each evolving point and its current time:

$$\hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_k + \frac{1}{K} v_\theta(\hat{\mathbf{x}}_k, k/K), \quad \hat{\mathbf{x}}_0 \sim \mathcal{N}(\mathbf{0}, I).$$

```

@torch.no_grad()
def integrate(field, start, steps=40):
    field.eval()
    state = start.clone()
    trajectory = [state.cpu()]
    for k in range(steps):
        t = torch.full((len(state),), k/steps, device=state.device)
        state = state + field(state, t)/steps
        trajectory.append(state.cpu())
    return torch.stack(trajectory)

start = torch.randn(1000, 2, device=device)
paths = integrate(velocity, start)
target_points = sample_data(1000).cpu()
fig, axes = plt.subplots(1, 4, figsize=(11, 2.8), layout="constrained")
points = [paths[0], paths[20], paths[-1], target_points]
for ax, xy, title in zip(axes, points, ["t = 0", "t = 0.5", "t = 1", "Data"]):
    ax.scatter(*xy.T, s=3, alpha=.6)
    ax.set(xlim=(-3, 3), ylim=(-3, 3), title=title, aspect="equal")
plt.show()

```

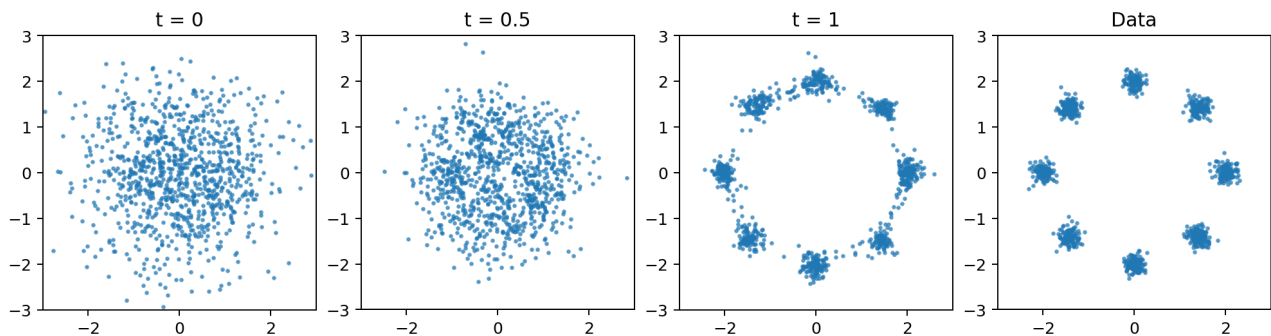


Figure 1: The 4,546-parameter field after 8,000 updates, using seed 3801 and batches of 128. The first three panels follow the same 1,000 points through 40 Euler steps; the fourth shows fresh data.

To see individual trajectories, join successive positions of a few particles:

```
plt.figure(figsize=(4, 4))
for i in range(20):
    plt.plot(paths[:, i, 0], paths[:, i, 1], alpha=.6)
plt.xlim(-3, 3); plt.ylim(-3, 3); plt.gca().set_aspect("equal")
plt.show()
```

Exercise

Repeat sampling with 5, 20 and 80 steps, reusing `start` each time. Compare the cluster widths and the points left between clusters. Why can these trajectories curve even though each training pair defines a straight path? As a further experiment, change the number or positions of the data clusters and train a fresh field.

From coordinates to spectrograms

For the points example, each state has two coordinates. A spectrogram contains thousands of real values, but the same path, regression loss and Euler update apply. We will now condition the velocity on a digit and voice, so that one network can generate several kinds of recording.

3 Represent sound as an image

We will generate someone saying a digit from one to nine. To use the image networks from the previous practicals, we first represent each recording as a *spectrogram*: an image showing how its sound changes over time. Reading from left to right follows the spoken word. Reading from bottom to top moves from low to high frequencies. Bright regions show which frequencies are strongest at each moment.

We train the flow on **the spectrogram image**, using the original audio recordings for listening. After generation, `to_audio` reconstructs a playable sound from the image; this conversion is approximate, and the function is included in the download.

Download `dl-practical7.py` [↗](#). On first use it downloads `audio.npz` [↗](#) from the research-data repository into the same working directory. Load the prepared images and recordings in IPython:

```
%run -n dl-practical7.py
images, digits, voices, recordings = audio_data("train")
print(images.shape)      # (540, 1, 128, 64)
print(recordings.shape)  # (540, 8064)
```

Axis of images	Meaning
540	Number of training recordings
1	One image channel: the strength at each frequency and time
128	Image rows: frequencies, from low to high
64	Image columns: positions from the start to the end of the clip

Thus `images[i, 0]` is one 128×64 image. Its display colours show the values in a single channel. Each corresponding recording contains 8,064 waveform values: about one second of sound at 8,000 samples per second.

Listen to a female recording of seven and display the image we will train on:

```
from IPython.display import Audio, display

i = torch.where((digits == 6) & (voices == 0))[0][0]
display(Audio(recordings[i].numpy(), rate=8000, normalize=False))
plt.figure(figsize=(5, 2.5))
plt.imshow(images[i, 0], origin="lower", aspect="auto",
            extent=(0, 1.008, 0, 4), cmap="magma")
plt.xlabel("Position in the word (seconds)")
plt.ylabel("Frequency (kHz)")
plt.show()
```

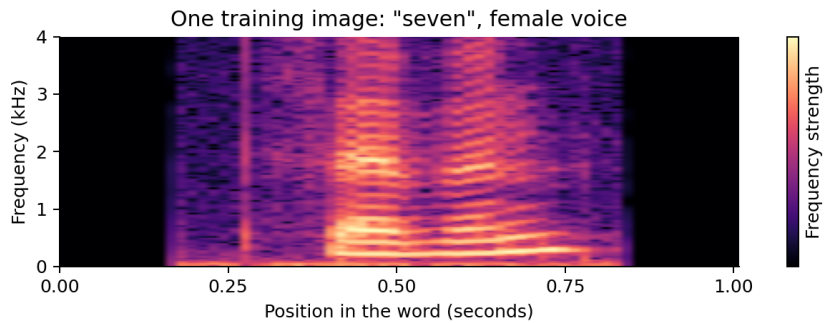


Figure 2: A female recording of seven, represented as one image. The flow is trained on the image values; the original waveform is used here for playback. Brightness shows frequency strength, and the colours simply make those values easier to see.

Exercise

Find another recording of seven, then one of three. Listen and compare their images: where does the speech begin and end, and where are the quiet intervals?

4 Condition on a digit and a voice

We want to choose which word the model says and which voice it uses. Each training image therefore comes with two labels: the spoken digit and the speaker. During generation, we choose these labels ourselves, for example *seven*, *female*, and pass them to the network at every flow step.

In the code, digits one–nine have labels 0–8, so seven has label 6. The female and male voices have settings 0 and 1. These labels select *embeddings*: small vectors learned alongside the convolutional weights. Training teaches the network how the requested word and voice relate to patterns in the spectrogram.

The network receives four inputs and predicts a velocity of the same size as the spectrogram:

Tensor	Shape for four recordings	Meaning
x	(4, 1, 128, 64)	Current noisy images
t	(4,)	Flow time for each recording
d	(4,)	Requested digit labels
s	(4,)	Requested voice settings
model(x, t, d, s)	(4, 1, 128, 64)	Predicted change to every image value

As in the points example, the output is a velocity. Each sampling step adds a small multiple of it to the image. Every step updates the whole image, including all positions within the spoken word.

The baseline stores 16 learned values for each digit and voice. Inspect the vectors for a female voice saying seven:

```
model = build_model().to(device)
d = torch.tensor([6], device=device)      # spoken digit 7
s = torch.tensor([0.], device=device)      # female voice
t = torch.tensor([.5], device=device)      # halfway through the flow

digit_vector = model.label(d)              # (1, 16): table row 6
voice_vector = model.voice_embedding(s)    # (1, 16): female table row
condition = model.time(t[:, None]) + digit_vector + voice_vector
print(condition.shape)                    # (1, 16)
```

The first convolution produces 16 feature maps. Each value in `condition` is added across one map, using `condition[:, :, None, None]` to broadcast it. Open Model and find this addition. It also appends two fixed coordinate grids to the image, so the convolutions can use frequency and audio-time position; this explains the three channels entering its first convolution.

Exercise

What information would be lost if every recording used the same digit embedding? Would increasing the embedding width also increase the length of the generated recording?

5 Train the velocity field

As in the previous practical, we train using corrupted images. Here a clean spectrogram $\mathbf{x}_1$ is mixed with Gaussian noise $\mathbf{x}_0$:

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1.$$

At $t = 0$ the image is entirely noise; increasing t reveals more of the recording. This is the same straight path used for the points example. Figure 3 shows one such path, with the label *seven, female* attached throughout.

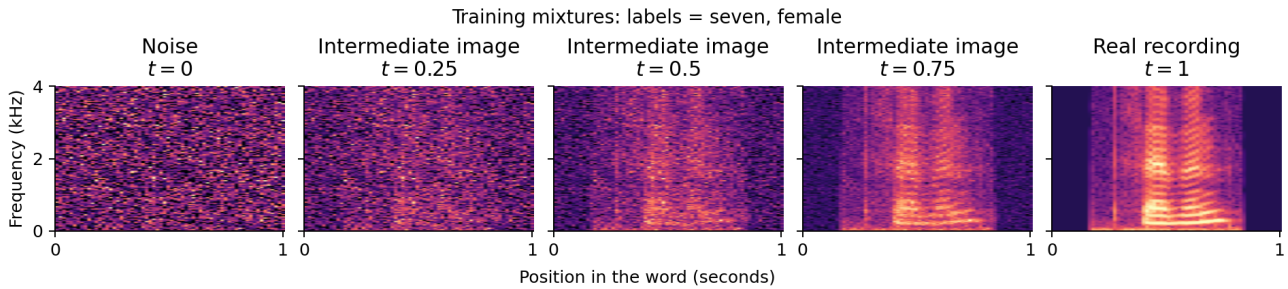


Figure 3: Training: mix a real spectrogram with noise to make examples at different flow times. Each example retains the recording's labels: seven, female. The network learns the velocity $\mathbf{x}_1 - \mathbf{x}_0$ from these mixtures.

For a batch of four recordings, form the corrupted images and ask the network to predict the velocity towards their clean versions:

```
x1 = images[:4].to(device)
d = digits[:4].to(device)
s = voices[:4].to(device)
x0 = torch.randn_like(x1)
t = torch.rand(4, device=device)
time = t[:, None, None, None] # (4, 1, 1, 1), for broadcasting
xt = (1-time)*x0 + time*x1
prediction = model(xt, t, d, s)
loss = F.mse_loss(prediction, x1-x0)
print(xt.shape, prediction.shape, loss.item())
```

The singleton dimensions in `time` let each recording's scalar t multiply all its image values. The target velocity is $\mathbf{x}_1 - \mathbf{x}_0$. The digit and voice labels come from the clean recording used to make that training example.

The supplied training loop repeats this calculation on fresh batches and updates the weights. Train the baseline from its initial seed:

```
torch.manual_seed(3801)
model = build_model().to(device)
history = train(model, device) # 3,000 updates
plt.plot(*zip(*history))
plt.xlabel("Update"); plt.ylabel("Flow matching loss")
plt.show()
```

Exercise

In the first block, set `t` to all zeros, then all ones. Check the states and target velocities. Why does the network need the current state and flow time as well as the requested digit and voice?

6 Generate a recording

To generate seven in the female voice, start with a fresh noise image and set the conditions to *seven, female*. At each step, the network receives the current image, flow time and these same conditions. Its predicted velocity gradually changes the noise into a spectrogram, which we then convert to sound.

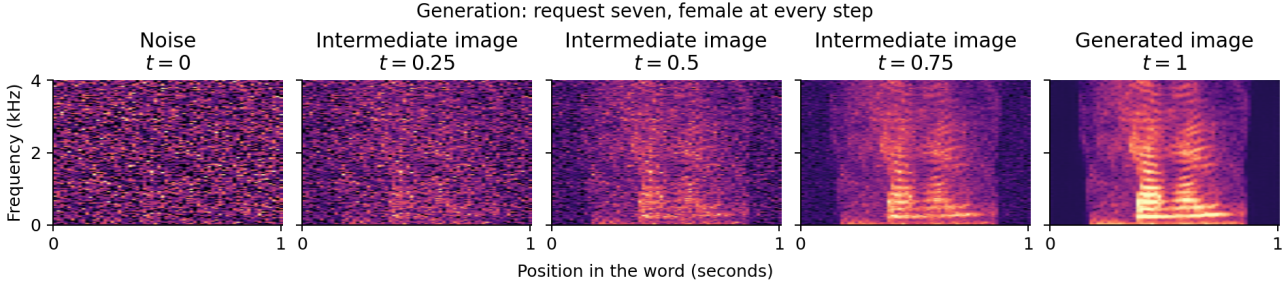


Figure 4: Generation: one noise image evolves through the trained baseline’s 64 flow steps. The digit and voice conditions are seven, female at every step. The last image is converted into playable audio. These are states from the learned flow; Figure 3 shows mixtures used to train it.

```
rng = torch.Generator(device=device).manual_seed(921)
start = torch.randn(1, 1, 128, 64, device=device, generator=rng)
d = torch.tensor([6], device=device) # spoken digit 7
s = torch.tensor([0.], device=device)
spectrogram = sample(model, d, s, start) # (1, 128, 64)
waveform = to_audio(spectrogram, rng) # (1, 8064)
display(Audio(waveform[0].cpu().numpy(), rate=8000))
```

`sample` performs 64 Euler updates, then removes the single channel dimension. `to_audio` converts the resulting 128×64 image into 8,064 waveform samples for playback. During generation, the evolving image and the chosen labels give the network all the information it uses; the clean recording is available only during training.

The program also learns a condition with the digit and voice labels hidden. Its sampler uses this to strengthen the effect of the requested labels, with fixed guidance strength 1.5. The extra embedding entries are reserved for this hidden condition. Keep these settings as supplied.

Exercise

Generate a different digit, then change the seed while keeping the labels fixed. Describe what each change controls.

7 Interpolate from female to male

To blend the voices, replace the voice vector by a mixture of the two learned vectors:

$$\mathbf{e}(s) = (1 - s)\mathbf{e}_F + s\mathbf{e}_M.$$

At $s = 0$ we use the female vector; at $s = 1$ we use the male vector. At $s = 0.5$ we use their average. `voice_embedding` performs this calculation before passing the condition to the network.

Keep these three quantities separate:

Audio time	Position within the spoken word; the spectrogram’s horizontal axis
Flow time t	Advances from noise at 0 to a generated spectrogram at 1
Voice setting s	Stays fixed while one recording is generated

For an interpolation, generate five complete recordings with $s = 0, 0.25, 0.5, 0.75, 1$. Each begins with the *same noise*. Within each recording, t advances from zero to one while s stays fixed.


```

start_five = start.repeat(5, 1, 1, 1)      # reuse one noise array
d = torch.full((5,), 6, device=device)      # five versions of seven
s = torch.linspace(0, 1, 5, device=device)
spectrograms = sample(model, d, s, start_five)
waveforms = to_audio(spectrograms, rng)
joined = F.pad(waveforms, (0, 1600)).flatten() # short gaps
display(Audio(joined.cpu().numpy(), rate=8000))

```

The five voices are played in order, female to male. Intermediate settings use mixtures of the learned embeddings; their sound depends on how the trained network responds between the two voices.

Exercise

Repeat with another digit. Then give each voice setting fresh noise instead of repeating `start`. Which version makes the effect of changing the voice easier to compare?

8 Improve the baseline

These examples use the baseline settings below and seed 3801. The score is measured on the validation recordings.

audio flow: 199,953 parameters, 3000 updates, batch 32; KAD x 1000 695.473.

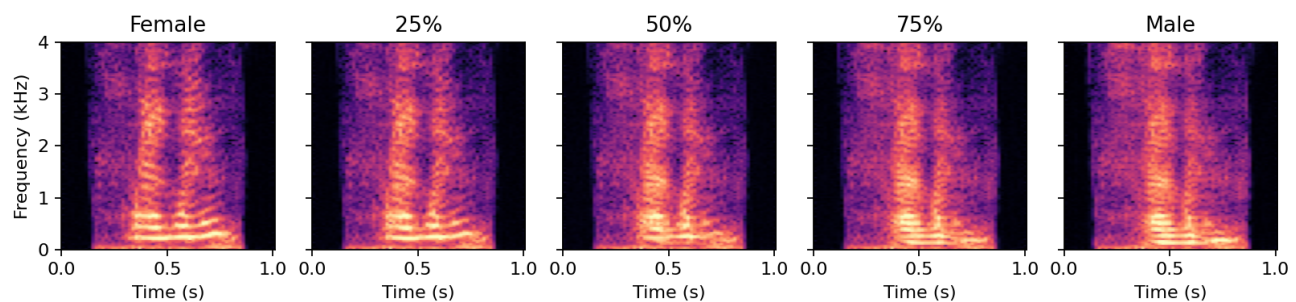


Figure 5: Five successive versions of seven, from female to male, using the same initial noise. These are the complete contents of `sample-7.wav`.

The complete program trains, evaluates and creates the submission ZIP. Improve its encoder-decoder within the budget: try residual blocks, conditioning in more layers, a richer time embedding, or a different learning-rate schedule. Compare several digits and both endpoint voices when judging a change.

KAD compares generated and real recordings through a frozen speech feature extractor. Lower is better. The program evaluates 360 generated female/male endpoint clips and averages the distances over the 18 voice-digit combinations. The nine submitted WAVs are short interpolation previews; intermediate voices have no matching real recordings for scoring. Evaluation needs TorchAudio for the same PyTorch version as your environment, and downloads its frozen evaluator on first use.

Further reading. AudioMNIST data; Flow Matching for Generative Modeling; Kernel Audio Distance. The data archive contains 540 training, 180 validation and 180 test recordings from the same two speakers, with different recordings in each split.

9 Submission

Download `dl-practical7.py` [↗](#), set `STUDENT_ID` and edit the model and training settings. Run the complete file or paste the whole program into one notebook cell. It trains, scores and creates the ZIP below; upload this to the corresponding Ultra item. The program downloads `audio.npz` [↗](#) from the research-data repository on first use.

```
lab-07-studentID.zip
lab-07-code.py
lab-07-results.json
samples/
  sample-1.wav
  sample-2.wav
  ...
  sample-9.wav
```

Details and constraints

- Save `sample-1.wav` through `sample-9.wav`, one per spoken digit. Each contains one five-clip transition from female to male, using the same initial noise at voice settings 0, 0.25, 0.5, 0.75 and 1. Keep the fixed 8 kHz mono PCM16 format and silent gaps.
- Use at most 1,000,000 parameters and buffer elements in total, 10,000 optimiser updates and a batch size of 64.
- Train from fresh weights on the 540 training spectrograms in `audio.npz`. Keep the two voices, digits 1–9 and recording splits fixed.
- Use no extra data, pretrained generators, distillation teacher, gradient accumulation or extra training batches per update. Reserve the fixed speech encoder for evaluation.
- Keep the flow loss, 10 percent label dropout, 64 Euler steps, guidance strength 1.5 and audio conversion fixed. KAD evaluates a separate fixed set of 360 generated endpoint clips against the 180 validation recordings. JSON stores raw KAD; the printed score is $\text{KAD} \times 1,000$.
- Put a brief abstract at the top of the Python file, with references and AI acknowledgements in comments. The program writes the JSON and sample files; submit these unchanged.

Marking

- Marks reflect the quality and simplicity of your solution and its measured performance within the constraints. Aim for a $\text{KAD} \times 1,000$ below 695.5.
- A pass requires a small improvement over the baseline. A 2:1 requires significant improvements using established techniques discussed in the course. A high first requires substantial improvements through well-implemented state-of-the-art methods, for example by integrating methods from recent papers, producing high-quality samples and scores within the budget.
- The exact marking function is withheld to avoid gamification.

Solutions for this assignment cannot later be publicised without request. Requests will be considered on a case-by-case basis after marking.